

Preguntas y Respuestas

TP Simulacion de Colas — Grupo 22
Sistema de Registro Dactilar — Municipalidad de Rio Cuarto

1. Arquitectura general del sistema

1.1 ¿Por que el programa esta dividido en dos archivos, simulacion.py y main.py? ¿Que ventaja tiene esa separacion?

La separacion sigue el principio de responsabilidad unica: simulacion.py contiene la logica pura de simulacion de eventos discretos sin saber nada de interfaces graficas, y main.py contiene la interfaz grafica sin saber como funciona la simulacion internamente. La ventaja es que se pueden modificar, testear o reemplazar de forma independiente. Por ejemplo, se podria cambiar toda la interfaz grafica sin tocar una sola linea de la simulacion, o ejecutar la simulacion desde un script de consola sin necesidad de PyQt5.

1.2 ¿Como se comunican simulacion.py y main.py? ¿Que informacion intercambian?

main.py importa simulacion.py como modulo. La comunicacion tiene tres momentos: primero main.py escribe los parametros del usuario directamente en las variables globales de simulacion.py (MEDIA_LLEGADA_EMP, MAX_COLA, etc.); despues llama a `resetear_estado()` y `simular(tiempo_max)`; y finalmente lee `sim.vector_estado` (la lista de filas generadas), `sim.reloj` (el tiempo final) y `sim.iteracion` (los eventos procesados). No hay mas comunicacion que esa.

1.3 ¿Que es vector_estado y en que formato esta?

Es una lista de diccionarios, uno por cada evento procesado. Cada diccionario representa una fila de la tabla y contiene el estado completo del sistema en ese momento: reloj, nombre del evento, estado de cada terminal, estado del tecnico, longitud de la cola, contadores estadisticos, numeros aleatorios generados y un snapshot de todos los empleados activos. Los valores son crudos, sin formatear: floats, enteros, booleanos y None. El formateo se hace en la GUI al momento de mostrar cada celda.

2. Simulacion de eventos discretos

2.1 ¿Que es una simulacion de eventos discretos y como se implementa en este programa?

Una simulacion de eventos discretos modela un sistema donde el estado cambia solo en momentos especificos llamados eventos. Entre dos eventos el sistema no cambia. En este programa los eventos son: llegada de empleado, fin de atencion, llegada del tecnico y fin de mantenimiento. Se mantiene una cola de eventos ordenada por tiempo (el heap), y en cada iteracion del loop se extrae el evento con menor tiempo, se avanza el reloj a ese tiempo y se procesa el evento.

2.2 ¿Por que se usa un heap binario para la cola de eventos en lugar de una lista comun?

Porque insertar y extraer el minimo en un heap cuesta $O(\log n)$, mientras que buscar el minimo en una lista no ordenada cuesta $O(n)$. Con 100.000 eventos, el heap hace esa operacion en unos 17 pasos en lugar de recorrer toda la lista. En la practica la diferencia es enorme: la simulacion corre en fracciones de segundo en lugar de varios segundos.

2.3 ¿Para que sirve el campo seq en cada evento del heap?

Para desempatar eventos que tienen el mismo tiempo. Cuando dos eventos coinciden en tiempo, Python compara el segundo campo de la tupla. seq es un contador monotono que crece con cada insercion, entonces el evento insertado primero tiene menor seq y sale primero del heap. Esto replica exactamente el comportamiento del antiguo `min()` lineal sobre una lista en orden de insercion, garantizando resultados reproducibles.

2.4 ¿Cuales son las condiciones de corte del loop principal?

El loop se detiene cuando ocurre cualquiera de estas condiciones: se procesaron 99.999 eventos (mas la fila de inicializacion = 100.000 filas totales); el reloj supera `tiempo_max` al inicio de una iteracion; el heap queda vacio; o el proximo evento a procesar tiene tiempo mayor a `tiempo_max`. En condiciones normales el corte ocurre por `tiempo_max`.

3. Variables aleatorias

3.1 ¿Que distribucion de probabilidad sigue la llegada de empleados y como se genera?

Distribucion exponencial negativa con media configurable (default 2 minutos). Se genera con la formula de la inversa: $t = -\text{media} * \ln(1 - \text{rnd})$, donde rnd es un numero aleatorio uniforme entre 0 y 1. Esta formula transforma una variable uniforme en una exponencial.

3.2 ¿Por que se trunca el RND ANTES de aplicar la formula?

Para que el numero mostrado en la tabla sea exactamente el que se uso en el calculo. Si se truncara despues, la tabla mostraria 0.26 pero la formula habria corrido con 0.2634, haciendo la tabla no auditable. Al truncar primero, el RND visible y el RND usado son identicos, lo que permite reproducir cualquier calculo a mano desde la tabla.

3.3 ¿Por que se trunca y no se redondea?

Porque el enunciado lo pide explicitamente. Truncar significa descartar los decimales sobrantes sin importar su valor. Redondear podria cambiar el ultimo decimal visible (si es 0.265, redondear da 0.27 pero truncar da 0.26). La funcion `trunc2` usa `int()` para garantizar truncado verdadero.

3.4 ¿Como se generan los tiempos de mantenimiento del tecnico? ¿Genera un solo RND por ronda o uno por terminal?

Genera un RND nuevo por cada terminal que mantiene. La funcion `gen_mantenimiento()` se llama dentro de `asignar_tecnico_a_terminal()`, que se ejecuta cada vez que el tecnico empieza a mantener una terminal. Esto significa que en una ronda completa de 4 terminales se generan 4 RNDs de mantenimiento distintos, cada uno con un tiempo diferente.

4. Logica del sistema

4.1 ¿Como funciona la prioridad del tecnico sobre los empleados?

El tecnico tiene prioridad pero no interrumpe atenciones en curso. Cuando una terminal se libera (por fin de atencion o fin de mantenimiento), la funcion `atender cola con terminal` verifica primero si el tecnico esta en estado 'Esperando Terminal Libre' y si esa terminal tiene `pendiente=True`. Si se cumplen ambas condiciones, el tecnico toma la terminal antes que cualquier empleado en la cola. Si el tecnico no esta esperando o la terminal no le corresponde, recien entonces se atiende el primer empleado de la cola.

4.2 ¿Por que el tecnico nunca entra a la cola de empleados?

Porque tiene su propio mecanismo de espera. Cuando llega y no hay terminal libre con `pendiente` pero hay terminales ocupadas con `pendiente`, pasa al estado 'Esperando Terminal Libre' y espera al costado. No compete con los empleados por un lugar en la cola. Esto es correcto porque su prioridad es mayor: en cuanto se libera una terminal que le corresponde, la toma antes que los empleados.

4.3 ¿Que es el flag pendiente de cada terminal y como funciona el ciclo del tecnico?

Es un booleano que indica si esa terminal todavia necesita mantenimiento en la ronda actual. Arranca en `True` para todas. Cuando el tecnico empieza a mantener una terminal, su `pendiente` pasa a `False`. Cuando termina la ronda (todas en `False`), se resetean todas a `True` y el tecnico descansa aproximadamente 60 minutos antes de volver. Asi el ciclo se repite indefinidamente.

4.4 ¿Como funciona el round-robin para asignar terminales a empleados?

Se mantiene un puntero global `ultimo_idx_terminal` que recuerda el indice de la ultima terminal asignada. Cuando llega un empleado, la busqueda empieza desde la siguiente posicion usando modulo para que sea circular. Esto distribuye la carga de forma equitativa entre las 4 terminales.

Es determinístico, no usa números aleatorios.

4.5 ¿Que pasa cuando un empleado llega y la cola esta llena?

Se rechaza (RT). Solo se incrementa el contador `contador_rt`. El empleado no entra al diccionario de empleados, no ocupa columna en la tabla y no vuelve. El enunciado dice que regresa a la media hora, pero el código solo reporta el porcentaje de rechazados sin programar su regreso. Esto es una desviación intencional del enunciado.

4.6 ¿En que momento exacto se calcula el tiempo de espera de un empleado?

En el momento exacto en que el empleado deja la cola y empieza a ser atendido, dentro de `atender_cola_con_terminal`. Se calcula como `reloj - hora_inicio_espera`, donde `hora_inicio_espera` es el reloj del momento en que entro a la cola. Para empleados atendidos de inmediato (sin pasar por la cola), la espera es 0 y se suma 0 al acumulador, pero se cuenta en `contador_espera` para que el promedio los incluya.

5. Estadísticas

5.1 ¿Como se calcula el porcentaje de empleados que se van?

Con la fórmula $\text{contador_rt} / \text{contador_llegaron} * 100$. `contador_rt` cuenta los empleados rechazados por cola llena. `contador_llegaron` cuenta todos los que llegaron al sistema, incluyendo los atendidos, los que esperaron y los rechazados. El cálculo se hace en la GUI al momento de mostrar la celda, no en `simulacion.py`.

5.2 ¿Como se calcula el promedio de espera y que empleados incluye?

Se calcula como $\text{acum_espera} / \text{contador_espera}$. `acum_espera` acumula la suma de los tiempos de espera de todos los empleados que fueron atendidos. `contador_espera` cuenta a todos los empleados atendidos, incluyendo los que esperaron 0 minutos por ir directo a una terminal libre. Los rechazados no entran en ninguno de los dos contadores. Esto hace que el promedio sea sobre todos los empleados del sistema, no solo los que esperaron.

5.3 ¿Por que los empleados atendidos de inmediato se cuentan en el promedio de espera con espera 0?

Porque el promedio pedido es el tiempo promedio de espera de todos los empleados del sistema, no solo de los que esperaron. Si un empleado llego y habia terminal libre, esperó 0 minutos, y eso debe reflejarse en el promedio. Excluirlos sobreestimaria el promedio real del sistema.

6. Estado AT del empleado

6.1 ¿Para que existe el estado AT si no esta en el modelo conceptual del problema?

AT es un estado de display, no de simulacion. Existe exclusivamente para que la tabla pueda mostrar en la fila de Fin Atencion que el empleado termino y se fue, antes de borrarlo del sistema. Si se borrara sin cambiar el estado, su columna quedaria vacia en esa fila como si nunca hubiera existido. Si se borrara dejando el estado SA, pareceria que sigue siendo atendido. AT resuelve ese problema: se cambia a AT, se guarda la fila (queda visible como x roja), y recien despues se borra. Ningun procesador toma decisiones basadas en si un empleado esta en AT.

6.2 ¿Por que el empleado se borra despues de guardar la fila y no antes?

Porque guardar_fila llama a snapshot_empleados, que lee el diccionario de empleados en ese momento. Si se borrara antes, el snapshot no lo incluiria y la columna del empleado quedaria vacia en esa fila. Al borrarlo despues, el snapshot captura el estado AT y la tabla muestra la x roja en la fila correcta.

7. Interfaz grafica

7.1 ¿Que es el renderizado virtual y por que es importante en este programa?

El renderizado virtual significa que la tabla no construye todas sus celdas de una vez, sino que solo renderiza las que son visibles en pantalla en cada momento. PyQt5 le pregunta al modelo celda por celda segun lo que el usuario puede ver. Con 100.000 filas, sin renderizado virtual se crearian millones de objetos visuales de una vez, consumiendo mucha memoria y haciendo la app lentisima. Con renderizado virtual, solo existen las celdas visibles.

7.2 ¿Por que la simulacion corre en un hilo aparte y no en el hilo principal?

Porque PyQt5 tiene un unico hilo principal que maneja la interfaz grafica. Si una operacion pesada corre en ese hilo, la ventana se congela y no responde hasta que termina. Al correr la simulacion en un hilo separado con QThread, la interfaz sigue respondiendo mientras simula, mostrando la barra de progreso y el mensaje de estado.

7.3 ¿Como se comunica el hilo de simulacion con el hilo principal cuando termina?

A traves de una senal de PyQt5 (pyqtSignal). Cuando la simulacion termina, el worker emite la senal finished. PyQt5 la entrega al hilo principal de forma segura, que ejecuta _on_sim_done para actualizar la UI. No se puede actualizar la UI directamente desde el hilo secundario porque PyQt5 lo prohíbe.

7.4 ¿Como funciona el panel de columnas congeladas?

Se usan dos tablas que comparten el mismo modelo de datos: `_frozen` a la izquierda con ancho fijo que muestra solo las 3 primeras columnas (`#`, `Reloj`, `Evento`), y `tbl_vector` a la derecha que oculta esas 3 columnas y tiene scroll horizontal completo. Ambas comparten el modelo de seleccion, por lo que seleccionar una fila en una la selecciona en la otra. El scroll vertical esta sincronizado en ambos sentidos. El resultado visual es que las 3 columnas siempre quedan visibles sin importar cuanto se haga scroll horizontal.

7.5 ¿Por que cambiar los filtros `j` e `i` no vuelve a simular?

Porque `j` e `i` son solo filtros de visualizacion sobre los datos ya calculados. Cuando termina la simulacion, todas las filas quedan en memoria en `self.todas_filas`. Cambiar `j` o `i` solo cambia que subconjunto de esas filas se le muestra al modelo. La busqueda de la fila de inicio se hace con `bisect_left` sobre una lista de relojes, que es $O(\log n)$ e instantaneo. No hay razon para correr la simulacion de nuevo.

7.6 ¿Por que hay dos `SimModel` distintos en la aplicacion?

Porque hay dos tablas con contenido diferente. `model_vector` es compartido por la tabla principal y el panel frozen, y siempre tiene las filas filtradas por `j` e `i`. `model_ultima` es exclusivo para la tabla de ultima fila y siempre tiene una sola fila: la ultima del vector de estado. Si compartieran el modelo, la tabla de ultima fila mostraria las mismas filas filtradas en lugar del estado final.

8. Decisiones de diseño

8.1 ¿Por que las columnas de empleados son dinamicas y no estaticas como las demas?

Porque la cantidad de empleados que aparecen en pantalla depende de que filas son visibles. Si se mostraran columnas para todos los empleados que llegaron en toda la simulacion (potencialmente cientos), la tabla tendria demasiadas columnas. Al calcularlas dinamicamente a partir de las filas visibles, solo aparecen los empleados relevantes para el rango actual, manteniendo la tabla manejable.

8.2 ¿Por que `guardar_fila` guarda valores crudos en lugar de valores formateados?

Por eficiencia. El loop de simulacion puede ejecutar 100.000 iteraciones. Si en cada una se construyeran strings formateados (truncados, SI/NO, porcentajes, promedios), se harian millones de operaciones de string que en su mayoria nunca se van a mostrar. Al guardar valores crudos y formatear solo las celdas visibles en la GUI, el loop es mucho mas rapido.

8.3 ¿Por que se uso `bisect` para filtrar filas por tiempo en lugar de un loop?

Porque `bisect_left` hace una busqueda binaria sobre la lista de relojes, encontrando la posicion correcta en $O(\log n)$. Sobre 100.000 filas eso son unos 17 pasos. Un loop lineal recorreria hasta 100.000 elementos. La diferencia es especialmente importante cuando el usuario cambia `j`

frecuentemente: con bisect el filtrado es instantaneo.

8.4 ¿Que problema resuelve el campo seq en el heap y cuando importa?

Resuelve el desempate cuando dos eventos tienen exactamente el mismo tiempo. Sin seq, Python intentaria comparar los strings del tipo de evento ('fin_atencion' vs 'llegada_emp') para ordenarlos, lo que podria dar resultados inesperados o incluso errores. Con seq, el evento insertado primero siempre sale primero cuando hay empate en tiempo, replicando el comportamiento del antiguo metodo lineal.

8.5 ¿Por que se decidio que el rechazado no regresa, si el enunciado dice que vuelve a los 30 minutos?

Fue una decision intencional del grupo. El enunciado pide reportar el porcentaje de empleados que se van, no modelar su regreso. Programar la re-llegada agregaria complejidad sin aportar a las metricas pedidas. El codigo solo incrementa contador_rt. Si la catedra lo objeta, la correccion es una linea: agregar push_evento(reloj + 30, 'llegada_emp') en la rama del rechazado.

9. Preguntas trampa

9.1 ¿El tecnico puede interrumpir una atencion en curso para hacer mantenimiento?

No. El tecnico tiene prioridad sobre los empleados en la cola de espera, pero nunca interrumpe una atencion que ya comenzo. Si llega y todas las terminales pendientes estan ocupadas, espera en estado 'Esperando Terminal Libre' hasta que alguna se libere naturalmente.

9.2 Si hay 5 empleados en la cola y llega uno mas, ¿entra o no entra?

No entra, se va. La condicion es `emp_en_cola() < MAX_COLA`, con `MAX_COLA = 5`. Si ya hay 5 esperando, la condicion es False y el empleado es rechazado. Esta es una desviacion intencional: el enunciado dice 'mas de 5', lo que permitiria hasta 6. El codigo rechaza cuando ya hay 5.

9.3 ¿Cuando se incrementa contador_espera, al llegar o al ser atendido?

Depende del caso. Para empleados atendidos de inmediato (sin pasar por la cola), se incrementa en `procesar_llegada_emp` en el momento de la llegada. Para empleados que esperaron en cola, se incrementa en `atender_cola_con_terminal` en el momento en que salen de la cola y empiezan a ser atendidos. En ambos casos el acumulador de espera recibe el valor correcto: 0 en el primer caso, `reloj - hora_inicio_espera` en el segundo.

9.4 ¿Por que trunc2 usa int() y trunc2_str usa math.floor con el parche 1e-9?

`trunc2` se usa en los generadores de variables aleatorias, donde los valores son resultados directos de formulas simples con RNDs frescos. Esos valores no tienen problemas de punto

flotante, y `int()` funciona correctamente. `trunc2_str` se usa en la GUI para truncar divisiones y acumulaciones (como el promedio de espera o el porcentaje de rechazados), que si pueden producir errores de punto flotante como 4.2999... en lugar de 4.30. El parche 1e-9 corrige ese error. Además, `trunc2` devuelve float porque el resultado se sigue usando en calculos, mientras que `trunc2_str` devuelve string porque va directo a la pantalla.

9.5 ¿Que pasa si el usuario presiona Simular dos veces seguidas rapidamente?

La segunda presion no hace nada. Al inicio de `on_simular` se verifica si `self._thread` esta corriendo. Si hay un hilo activo, la funcion retorna inmediatamente sin crear un nuevo hilo ni una nueva simulacion.

9.6 ¿Por que el campo tipo fue eliminado de la cola de empleados?

Porque era un campo residual de una version anterior del codigo donde la cola podia contener tanto empleados como el tecnico. Cuando se decidio que el tecnico tuviera su propio mecanismo de espera, el campo tipo quedo sin uso: ningun lugar del codigo lo leia. Se simplifico la cola para que guarde directamente el `emp_id` en lugar de un diccionario con tipo e id.

9.7 ¿En la fila de Fin Mantenimiento puede aparecer un RND de atencion? ¿Por que?

Si. Cuando el tecnico termina de mantener una terminal y la libera, se llama a `atender_cola_con_terminal` para resolver si hay empleados esperando. Si habia alguien en la cola, ese empleado se asigna a la terminal y se genera un RND de atencion. Ese RND se guarda en la fila de Fin Mantenimiento con `rnds.update()`. Antes de este fix el RND se generaba pero se descartaba, haciendo el tiempo de fin de atencion aparecer sin el numero que lo origino.

10. Preguntas sobre el codigo — Python

10.1 ¿Para que sirve la palabra global en Python y por que se usa en los procesadores de eventos?

En Python, si dentro de una funcion se intenta modificar una variable que existe fuera de ella, Python la trata como variable local y tira error. La palabra `global` le dice a Python que esa variable es la global y no una nueva local. En los procesadores se usa porque modifican variables como `contador_llegaron`, `contador_rt`, `reloj`, etc. que viven fuera de la funcion. Sin `global`, Python crearia una variable local nueva en lugar de modificar la global.

```
def procesar_llegada_emp():
    global id_emp_contador, contador_llegaron # sin esto, error al hacer +=
    id_emp_contador += 1
    contador_llegaron += 1
```

10.2 ¿Que hace next() y por que se usa con una expresion generadora?

`next()` toma un iterador o generador y devuelve su primer elemento, luego para. La expresion generadora `t for t in terminales if t['id'] == terminal_id` recorre la lista de terminales y filtra las que cumplen la condicion, pero no construye una lista completa: genera los resultados de a uno. `next()` toma el primero y para, sin recorrer el resto. Es equivalente a un `for` con `break` pero en una sola linea.

```
# Equivalente largo:
for t in terminales:
    if t['id'] == terminal_id:
        term = t
        break

# Con next():
term = next(t for t in terminales if t['id'] == terminal_id)
```

10.3 ¿Que hace el operador % (modulo) en el round-robin y por que hace la busqueda circular?

El modulo devuelve el resto de la division entera. En el round-robin, `idx = (ultimo_idx_terminal + offset) % n` hace que el indice nunca supere `n-1`. Por ejemplo con 4 terminales (`n=4`): si ultimo fue 3 (terminal 4) y offset es 1, $(3+1) \% 4 = 0$, volviendo a la terminal 1. Si ultimo fue 2 y offset es 3, $(2+3) \% 4 = 1$, terminal 2. El modulo hace que la busqueda sea circular automaticamente sin necesidad de `if`.

```
n = 4 # 4 terminales
(3 + 1) % 4 = 0 # despues de la terminal 4, va a la 1
(2 + 3) % 4 = 1 # salta de la 3 a la 2
(0 + 1) % 4 = 1 # de la 1 a la 2
```

10.4 ¿Que hace `any()` en `hay_pendiente_ocupada()` y por que es mejor que un `for`?

`any()` recorre un iterable y devuelve `True` en cuanto encuentra el primer elemento `True`, sin seguir recorriendo el resto. Es un cortocircuito: si la primera terminal ya cumple la condicion, no evalua las demas. Un `for` recorreria todas sin esa optimizacion. Es mas legible y eficiente.

```
# Con any():
return any(t['pendiente'] and t['estado'] == 'Ocupada' for t in terminales)

# Equivalente con for:
for t in terminales:
    if t['pendiente'] and t['estado'] == 'Ocupada':
        return True
return False
```

10.5 ¿Que diferencia hay entre `dict.get(key)` y `dict[key]`?

`dict[key]` tira `KeyError` si la clave no existe. `dict.get(key)` devuelve `None` si la clave no existe, sin tirar error. En el modelo se usa `f[emp'].get(a)` porque el empleado con ese ID puede no estar en el snapshot de esa fila (por ejemplo si ya fue borrado o si todavia no habia llegado). Con `dict[key]` tiraria error cada vez que el empleado no existe en esa fila.

```
tup = f['emp'].get(a) # devuelve None si el empleado no esta
if tup is None: return '' # celda vacia, sin error
```

10.6 ¿Que hace rnds.update(extra) y que pasa si extra es un dict vacio?

dict.update() agrega todas las claves y valores del dict argumento al dict original, pisando las existentes si hay conflicto. Si extra es {} (dict vacio), update no hace nada: el dict original queda igual. Esto es exactamente lo que necesitamos: si atender_cola_con_terminal no asigno a nadie devuelve {}, y rnds queda sin cambios.

```
rnds = {'llegada_tec': 0.54, 't_llegada_tec': 59.3}
rnds.update({'atencion': 0.31, 't_atencion': 5.93})
# rnds ahora tiene las 4 claves

rnds.update({}) # no cambia nada
```

10.7 ¿Por que cola.pop(0) y no cola.pop()? ¿Que diferencia hay?

pop() sin argumento saca el ultimo elemento. pop(0) saca el primero. La cola de empleados funciona como FIFO (primero en llegar, primero en ser atendido): el primer empleado en entrar a la cola debe ser el primero en salir. Por eso se saca el indice 0. Una alternativa mas eficiente seria usar collections.deque con popleft(), pero para el tamaño de cola que maneja este sistema (maximo 5 elementos) la diferencia es irrelevante.

10.8 ¿Que es una list comprehension y donde se usa en el codigo?

Es una forma compacta de construir una lista aplicando una expresion a cada elemento de un iterable, con un filtro opcional. En el codigo se usan en varios lugares, por ejemplo en guardar_fila para capturar el estado de las 4 terminales de una vez:

```
'term_est': [t['estado'] for t in terminales] # ['Libre', 'Ocupada', 'Libre', 'Libre']
'term_pend': [t['pendiente'] for t in terminales] # [True, False, True, True]
'term_fin': [t['fin_aten'] for t in terminales] # [None, 74.3, None, None]
```

10.9 ¿Que es una dict comprehension y donde se usa?

Es lo mismo que una list comprehension pero construye un diccionario. Se usa en snapshot_empleados para construir el snapshot de todos los empleados activos en una sola linea:

```
return {
    emp['col']: (emp['estado'], emp['hora_inicio_espera'], emp['terminal_id'])
    for emp in empleados.values()
}
```

La clave es el numero de columna del empleado y el valor es una tupla con sus tres datos. Es equivalente a un for que construye el dict de a uno pero mas compacto.

10.10 ¿Que hace bisect.bisect_left y como funciona internamente?

bisect_left hace una búsqueda binaria sobre una lista ordenada y devuelve el índice donde debería insertarse el valor para mantener el orden. Si el valor ya existe, devuelve el índice del primero. Internamente divide la lista por la mitad en cada paso: si el valor buscado es mayor que el elemento del medio, busca en la mitad derecha; si es menor, en la izquierda. Repite hasta encontrar la posición. Cuesta $O(\log n)$: sobre 100.000 elementos, máximo 17 pasos.

```
relojes = [0.0, 1.34, 2.71, 5.02, 8.43, ...]
bisect.bisect_left(relojes, 5.0)
# devuelve 3: la primera fila con reloj >= 5.0 esta en el índice 3
```

11. Preguntas sobre el código — PyQt5

11.1 ¿Que métodos obligatorios hay que implementar al heredar de QAbstractTableModel?

Son tres métodos mínimos obligatorios: rowCount() que devuelve la cantidad de filas, columnCount() que devuelve la cantidad de columnas, y data() que devuelve el contenido de una celda dado un índice y un rol. Sin estos tres PyQt5 no puede mostrar nada. headerData() es opcional pero necesario para mostrar los títulos de columnas.

```
def rowCount(self, parent=None):    return len(self.rows)
def columnCount(self, parent=None): return len(self._specs)
def data(self, index, role):        # devuelve texto o color segun role
def headerData(self, section, orientation, role): # títulos
```

11.2 ¿Que es un role en PyQt5 y cuales se usan en SimModel?

Un role es un identificador que le dice al modelo que tipo de información se está pidiendo para una celda. No siempre se quiere el texto: a veces se quiere el color, el tooltip, la fuente, etc. En SimModel se usan dos roles: Qt.DisplayRole para el texto que se muestra en la celda, y Qt.ForegroundRole para el color del texto. Cada vez que PyQt5 necesita renderizar una celda, llama a data() con cada role por separado.

11.3 ¿Para que sirven beginResetModel y endResetModel en set_content?

Le avisan a PyQt5 que los datos del modelo van a cambiar completamente. beginResetModel le dice a la vista que se prepare (descarta índices internos y selecciones). endResetModel le dice que ya terminó y que redibuje todo. Sin estas llamadas, la vista podría tener índices desactualizados y mostrar datos incorrectos o crashear al intentar acceder a filas que ya no existen.

11.4 ¿Que es pyqtSignal y como funciona el mecanismo de señales en PyQt5?

pyqtSignal define una señal en una clase que hereda de QObject. Una señal es un evento que puede ser emitido con emit() y al que otros objetos pueden conectarse con connect(). Cuando se emite, PyQt5 llama automáticamente a todos los métodos conectados. En el código, finished =

`pyqtSignal()` define la señal, `self.finished.emit()` la emite al terminar la simulacion, y `self._worker.finished.connect(self._on_sim_done)` conecta esa señal al metodo que actualiza la UI.

11.5 ¿Por que se usa `moveToThread` en lugar de subclasear `QThread` directamente?

Es el patron recomendado por Qt. Subclasear `QThread` y sobrescribir `run()` funciona pero tiene problemas: el objeto vive en el hilo principal y solo `run()` corre en el secundario, lo que genera confusion con las senales. Con `moveToThread`, el objeto completo (incluyendo sus slots) vive en el hilo secundario, haciendo la comunicacion por senales mas segura y predecible.

11.6 ¿Que hace `deleteLater` y por que no se usa del directamente?

`deleteLater` le dice a PyQt5 que borre el objeto cuando el event loop vuelva al hilo que lo posee y no haya operaciones pendientes. del en Python borra la referencia inmediatamente, pero si PyQt5 todavia tiene pendiente alguna operacion con ese objeto (como procesar una señal), puede crashear. `deleteLater` es la forma segura de liberar objetos de Qt.

11.7 ¿Por que ambas tablas (frozen y principal) tienen `ScrollBarAlwaysOn` horizontal?

Para que los dos viewports tengan exactamente la misma altura. Cuando una scrollbar esta visible ocupa espacio fisico abajo del viewport. Si una tabla tiene la scrollbar visible y la otra no, sus viewports tienen alturas distintas y el rango de scroll vertical no coincide, produciendo un desfase al llegar al final. Al forzar `ScrollBarAlwaysOn` en las dos, ambas reservan el mismo espacio abajo y sus alturas coinciden siempre. La scrollbar del frozen queda inerte porque sus 3 columnas entran sin scroll.

11.8 ¿Que hace `setSelectionModel` y por que se comparte entre las dos tablas?

`setSelectionModel` asigna el objeto que gestiona que filas estan seleccionadas. Al compartirlo entre `_frozen` y `tbl_vector`, cuando el usuario hace click en una fila de cualquiera de las dos tablas, ambas resaltan esa misma fila. Sin esto, seleccionar en una no afectaria a la otra y visualmente pareceria que son tablas independientes.

11.9 ¿Que hace `paintEvent` en `GroupedHeader` y cuando lo llama PyQt5?

`paintEvent` es el metodo que PyQt5 llama cada vez que necesita redibujar el widget: cuando aparece en pantalla por primera vez, cuando el usuario hace scroll, cuando se redimensiona la ventana, o cuando se llama a `viewport().update()`. Al sobrescribirlo en `GroupedHeader` se reemplaza el dibujado estandar de `QHeaderView` por el dibujado personalizado de dos niveles usando `QPainter`.

11.10 ¿Que hace `uic.loadUi` y que ventaja tiene sobre construir la UI en Python?

uic.loadUi carga un archivo .ui (XML generado por Qt Designer) y crea todos los widgets definidos ahí como atributos del objeto receptor. La ventaja es que la estructura visual se define en Qt Designer de forma gráfica, sin escribir código. Si se quiere mover un botón o cambiar un layout, se hace en el diseñador y no hay que tocar el código Python. Separa el diseño visual de la lógica.